

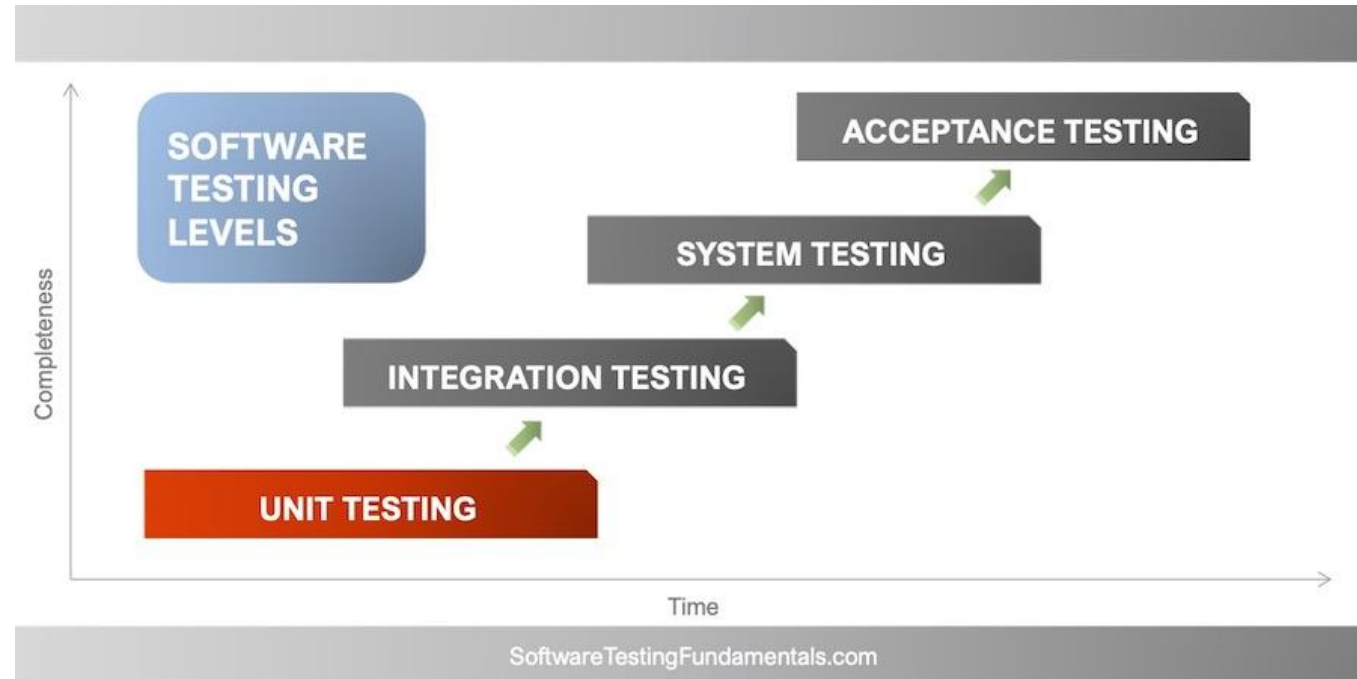
Software Testing

Course's Code: CSE 453

Levels of Software Testing and Testing Pyramid
(Chapter 2)

Test Levels

- The term Test Level provides an indication of the focus of the testing, and the types of problems it is likely to uncover.
- The typical levels of testing are:
 - ☐ Unit Testing
 - ☐ Integration Testing
 - ☐ System Testing
 - ☐ Acceptance Testing



Unit Testing

- Test each module individually
- A unit or a module is the smallest part of an application like functions/procedures, classes, interfaces
- Unit tests are usually written and run by software developers

Why Unit Testing?

- Unit tests help to fix bugs early in the development cycle and save costs.
- It helps the developers to understand the code base and enables them to make changes quickly
- Good unit tests serve as project documentation
- Unit tests help with code re-use. Migrate both your code and your tests to your new project. Tweak the code until the tests run again.

Unit Testing

When is Unit Testing performed?

- Unit Testing is the first level of software testing and is performed prior to Integration Testing.
- Though unit testing is normally performed after coding, sometimes, specially in test-driven development (TDD), automated unit tests are written prior to coding.

Who performs it?

- It is normally performed by software developers themselves or their peers.
- In rare cases, it may also be performed by independent software testers but they will need to have access to the code and have an understanding of the architecture and design.

Integration Testing

- Once the units have been written, the next stage is to put them together to create the system
 - ❑ This called integration
- The purpose of integration testing is to expose defects in the interfaces and in the interactions between integrated components or systems
- Test bases for integration testing can include
 - ❑ the software and the system design
 - ❑ a diagram of the system architecture
 - ❑ Workflows and use cases
- It is done by integration testers or test team

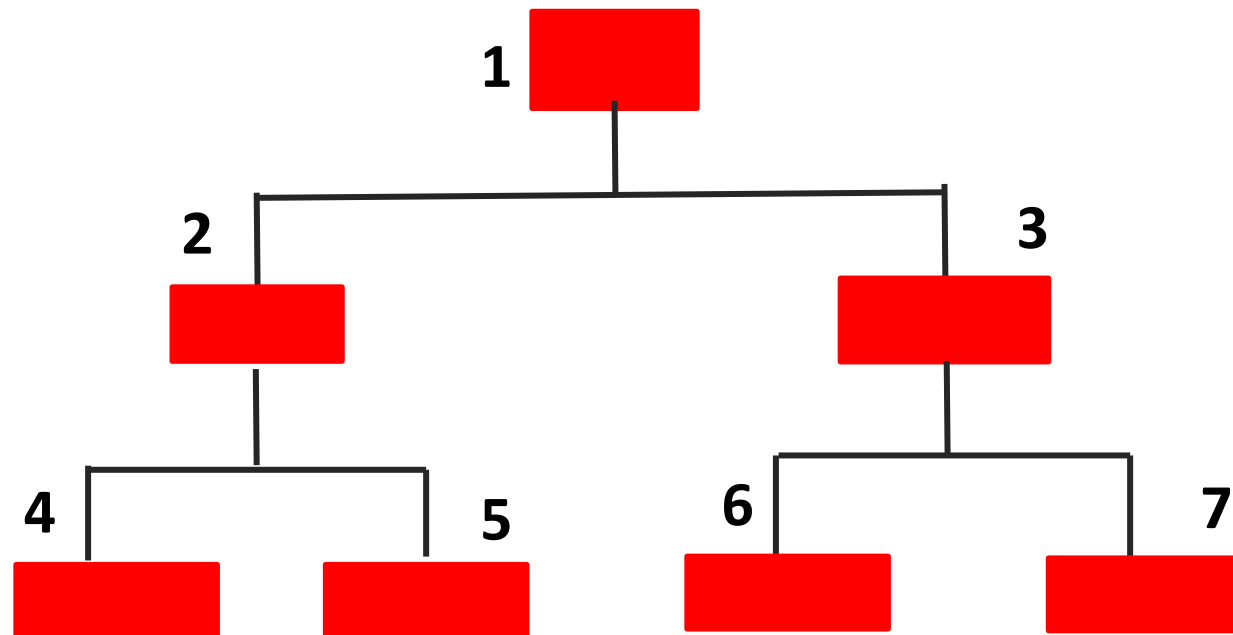
Integration Testing

- There are three commonly quoted integration testing
 - ☐ Big-bang Integration
 - ☐ Top-down integration
 - ☐ Bottom-up Integration
- Big-bang Integration
 - ☐ All units are linked at once, resulting in a complete system
 - ☐ It is difficult to isolate any errors found
 - ☐ regarded as poor choice
 - ☐ introduces risks, that problems may be discovered late in the project

Integration Testing

➤ Top-down Integration

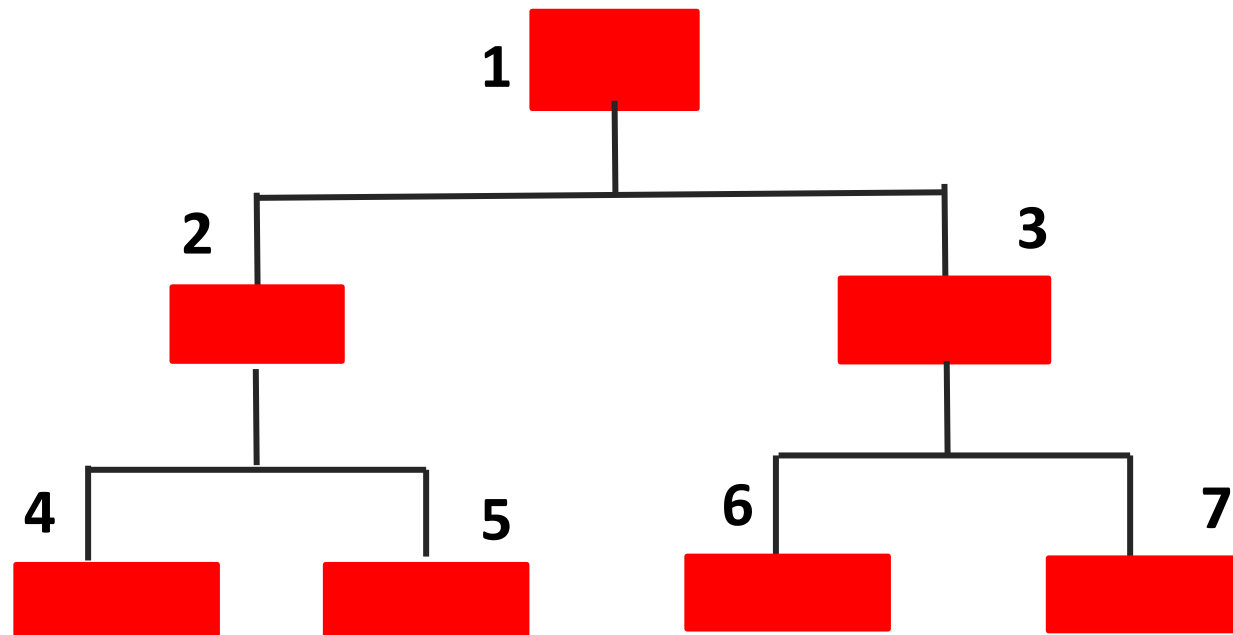
- ❑ The system is built in stages, starting with components that call other components
- ❑ Components that call others are usually placed above those that are called
- ❑ permits the testers to evaluate component interfaces, starting with those at the top



Integration Testing

➤ Top-down Integration

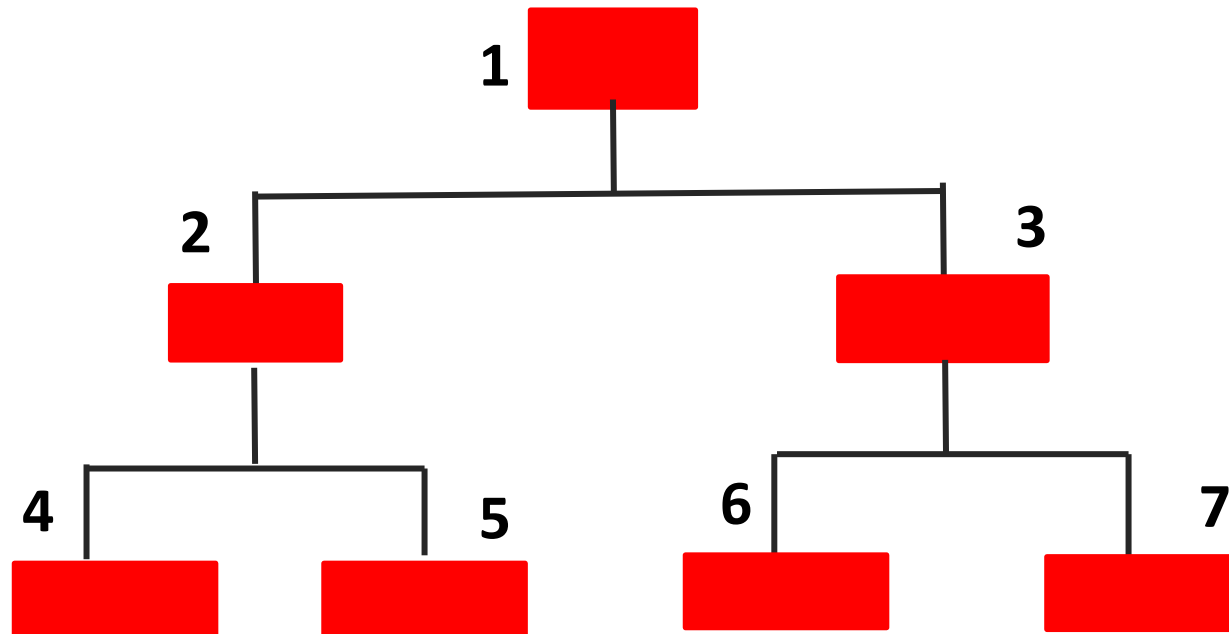
- ☐ It requires that the interactions of each component must be tested when it is built
- ☐ Sometimes, those lower down in the hierarchy may not have been built or integrated yet
- ☐ creating a skeleton implementation of the component, that is not have been built or integrated yet, called a stub



Integration Testing

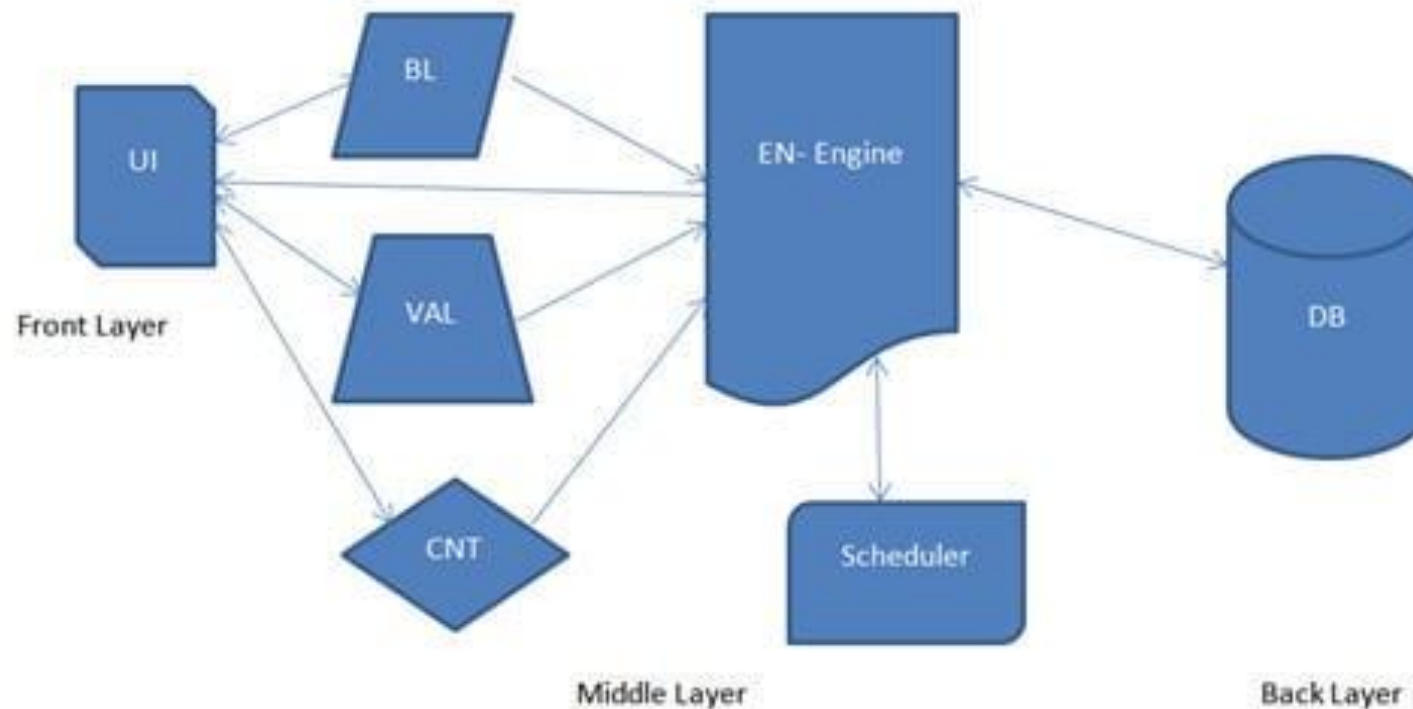
➤ Bottom-up Integration

- ❑ This is the opposite of Top-down Integration and components are integrated in bottom –up manner
- ❑ In this case, the components that may not be in place are replaced by specially written components
- ❑ Those are called drivers



Example of Integration Testing

- I am the owner of an advertising company and I post ads on different websites. At the end of the month, I want to see how many people saw my ads and how many people clicked on my ads. I need a report for my ads displayed and I charge accordingly to my clients.
- GenNext software developed this product for me and below was the architecture:



Example of Integration Testing

- **UI** – User Interface module, which is visible to the end user, where all the inputs are given.
 - **BL** – Is the Business Logic module, which has all the all the calculations and business specific methods.
 - **VAL** – Is the Validation module, which has all the validations of the correctness of the input.
 - **CNT** – Is the content module which has all the static contents, specific to the inputs entered by the user. These contents are displayed in the reports.
 - **EN** – Is the Engine module, this module reads all the data that comes from BL, VAL and CNT module and extracts the SQL query and triggers it to the database.
 - **Scheduler** – Is a module which schedules all the reports based on the user selection (monthly, quarterly, semiannually & annually)
 - **DB** – Is the Database.
- Now, having seen the architecture of the entire web application, as a single unit, Integration testing, in this case, will focus on the flow of data between the modules.

Example of Integration Testing

- In this example, the communication of data is done in an XML format. So whatever data the user enters in the UI, it gets converted into an XML format.
- In our scenario, the data entered in the UI module gets converted into XML file which is interpreted by the 3 modules BL, VAL and CNT. The EN module reads the resultant XML file generated by the 3 modules and extracts the SQL from it and queries into the database. The EN module also receives the result set and converts it into an XML file and returns it back to the UI module which converts the results in user readable form and displays it.
- In the middle we have the scheduler module which receives the result set from the EN module, creates and schedules the reports.

Example of Integration Testing

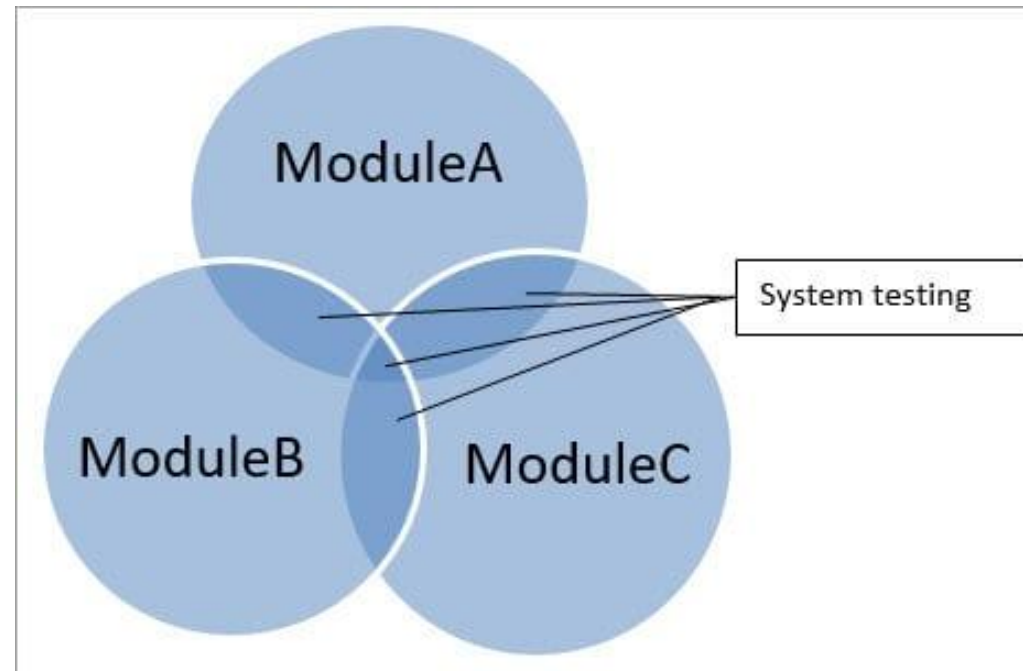
- So where Integration testing does comes into the picture?
- Well, testing whether the information/data is flowing correctly or not will be your integration testing, which in this case would be validating the XML files. Are the XML files generated correctly? Do they have the correct data? Are the data is being transferred correctly from one module to another? All these things will be tested as part of Integration testing.

System Testing

- System testing is conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements
- System testing is most often the final test to verify that the system to be delivered meets the specification and its purpose
- It is usually carried out by a team that is independent of the development process
- **System testing** should investigate **both functional and non-functional requirements** of the testing
- Functional requirements provide detail on what the application being developed will do
- **Non-functional system testing** looks at those aspects that are important but not directly related to what functions the system performs. For example, **Load Handling test , Performance test etc.**

System Testing

- If an application has three modules A, B, and C, then testing done by combining the modules A & B or module B & C or module A& C is known as Integration testing. Integrating all the three modules and testing it as a complete system is termed as System testing.



System Testing

➤ **When is it performed?**

- ❖ System Testing is the third level of software testing performed after Integration Testing and before Acceptance Testing.

➤ **Who performs it?**

- ❖ Normally, independent software testers perform System Testing.

➤ **Why System Testing?**

- ❖ #1) It is very important to complete a full test cycle and ST is the stage where it is done.
- ❖ #2) ST is performed in an environment that is similar to the production environment and hence stakeholders can get a good idea of the user's reaction.
- ❖ #3) It helps to minimize after-deployment troubleshooting and support calls.
- ❖ #4) In this STLC stage Application Architecture and Business requirements, both are tested.

Types of System Testing

➤ Types

□ **System Testing** is the most comprehensive level of testing and many types of tests are performed.

Some of them are mentioned below:

- ❖ **Smoke Testing** to ensure that the most important functions work and to decide whether the build is fit for further testing.
- ❖ **Functional Testing** to ensure that features work as per the functional requirements.
- ❖ **Regression Testing** to ensure that changes (enhancements or defect fixes) to the software have not adversely affected it.
- ❖ **Usability Testing** to determine if the system is easily usable from an end-user's perspective.
- ❖ **Performance Testing** to determine how the system performs in terms of responsiveness and stability under a certain load.
- ❖ **Security Testing** to uncover vulnerabilities of the system and determine that its data and resources are protected from possible intruders.
- ❖ **Compliance Testing** to determine the compliance of the system with internal or external standards.

Examples of System Testing

➤ Examples

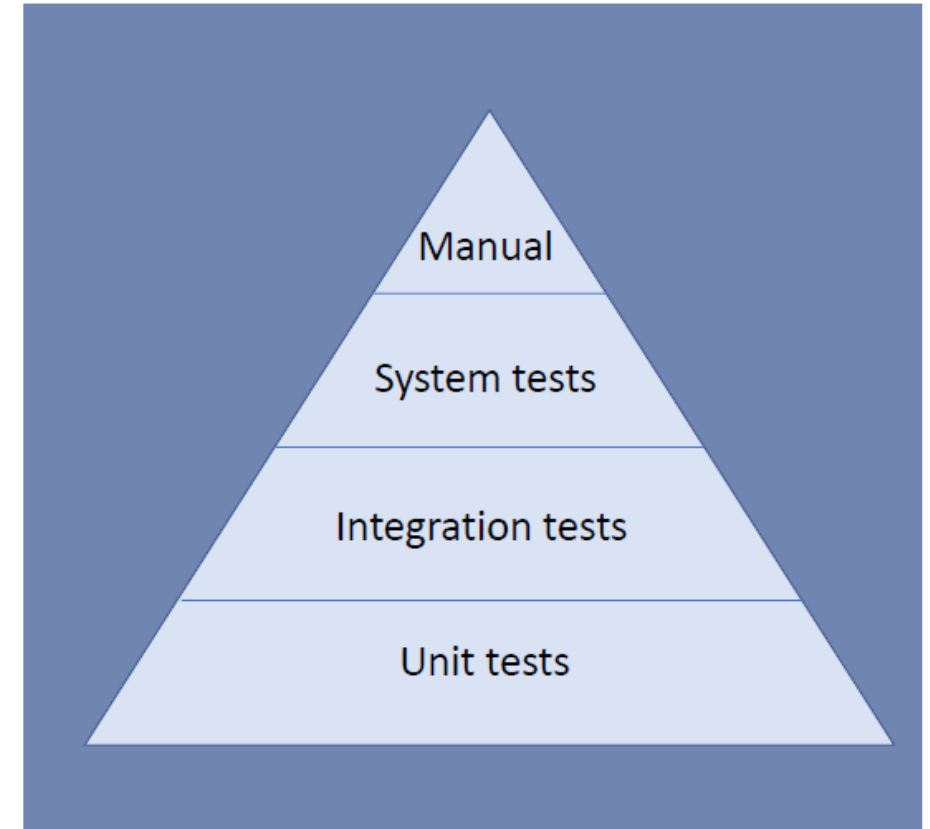
- ☐ What if an online transaction fails after confirmation?
- ☐ What if an item placed in a cart of an online site does not allow to place an order?
- ☐ What if the system crashes when a load is increased on the system?
- ☐ What if the system crashes and is not able to recover the data as desired?
- ☐ What if installing software on the system takes much more time than expected and at the end gives an error?
- ☐ What if a website response time increases much more than expected after enhancement?
- ☐ What if a website becomes too slow that the user is unable to book his/her travel ticket?

Acceptance Testing

- **The next step after system testing is often acceptance testing**
 - ❖ The purpose is to provide the end users with confidence that the system will function according to their expectations
 - ❖ Usually done by users or customers, other stakeholders may be included
- **Types of acceptance testing**
 - ❖ **Alpha Testing**: It takes place at the developer's site before release to external customers
 - ❑ Done by potential users/customers or an independent test team at the developer's site
 - ❖ **Beta Testing**: It takes place at the customer's site. Tested by a group of customers, who use the product at their own locations
 - ❑ Provide the feedback, before the system is released. Often known as “field testing”

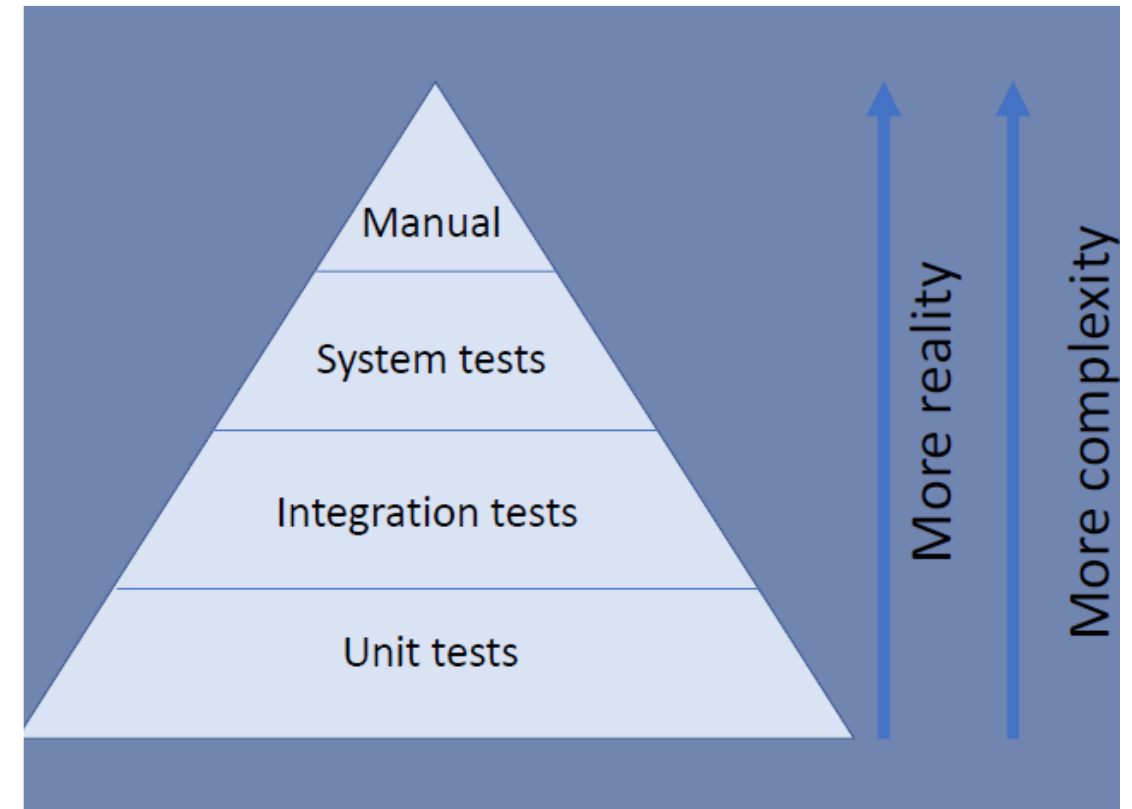
Testing Pyramid

- The "Test Pyramid" is a visual aid that tells us to group software tests into buckets (Unit Testing , Integration Testing and System Testing) of different granularity.
- It also gives an idea of how we should balance between those groups.
- The idea of Testing Pyramid is
- that you do a lot of unit tests and that is why this is the basis of the pyramid.
- Then you go for integration tests, and there you do a little bit less, because they are costly.
- And then you do system tests but also little bit of less system tests.
- And then finally manual tests where you try to avoid as much as possible, because they are costly.



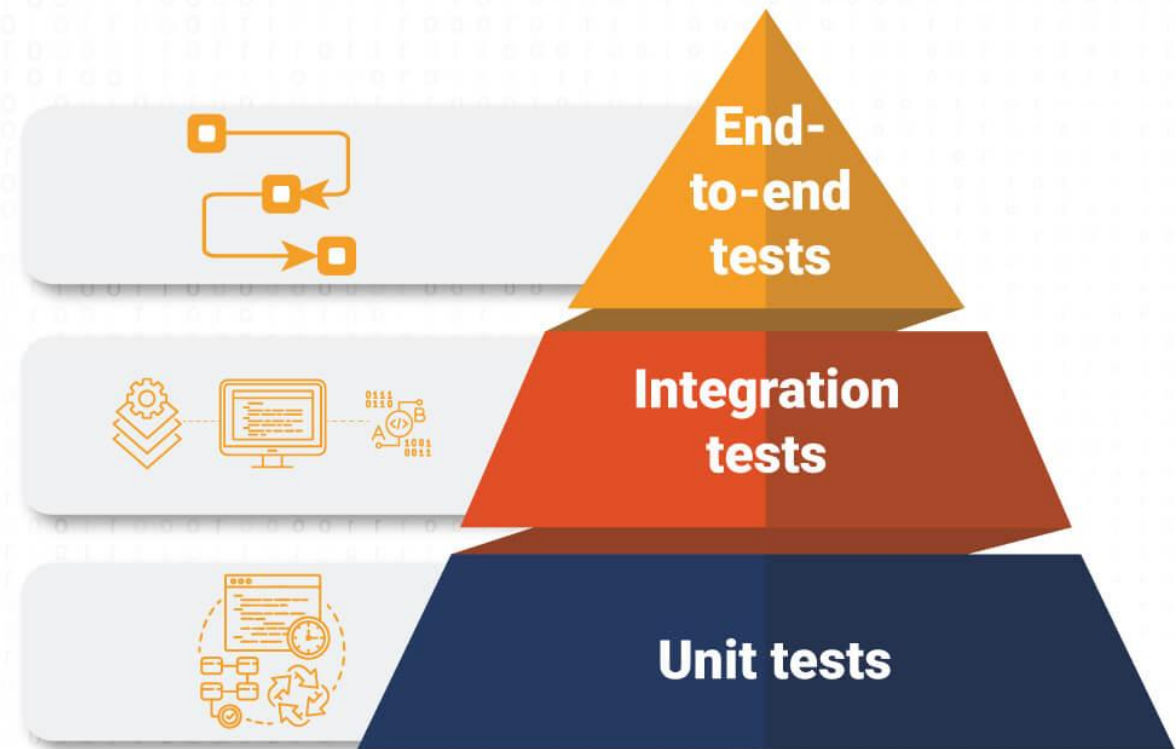
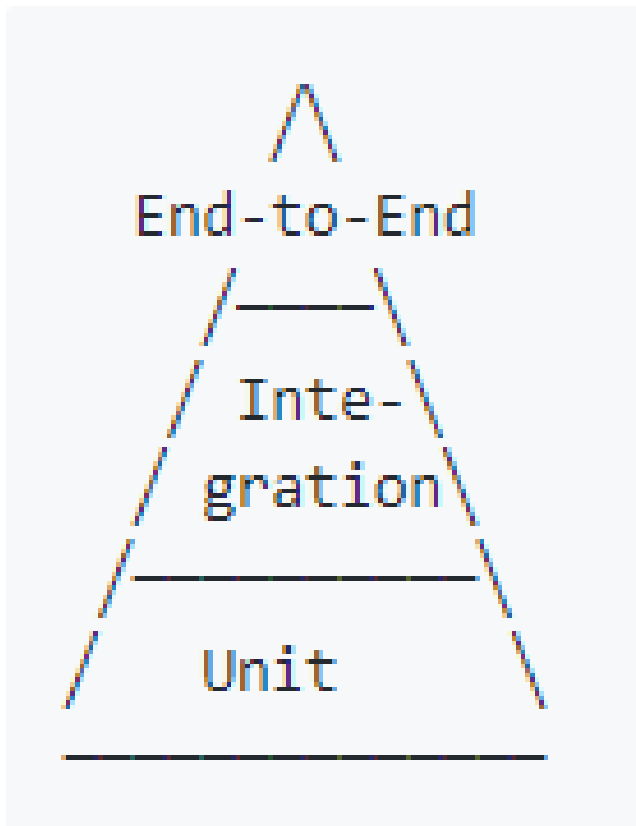
Testing Pyramid

- Whenever you have a business rule, meaning an algorithm that you can very well exercise, do unit tests because it's quite easy to control and quite easy to exercise the different cases that your algorithm may encounter.
- And then as soon as your system starts to talk to external system, for example, a database involved, then you can test interaction between the component of your system that talks to the database and database together. This is an integration test.
- And then for the system test, system test are very costly so we usually in practice don't retest the whole system from the system test perspective.

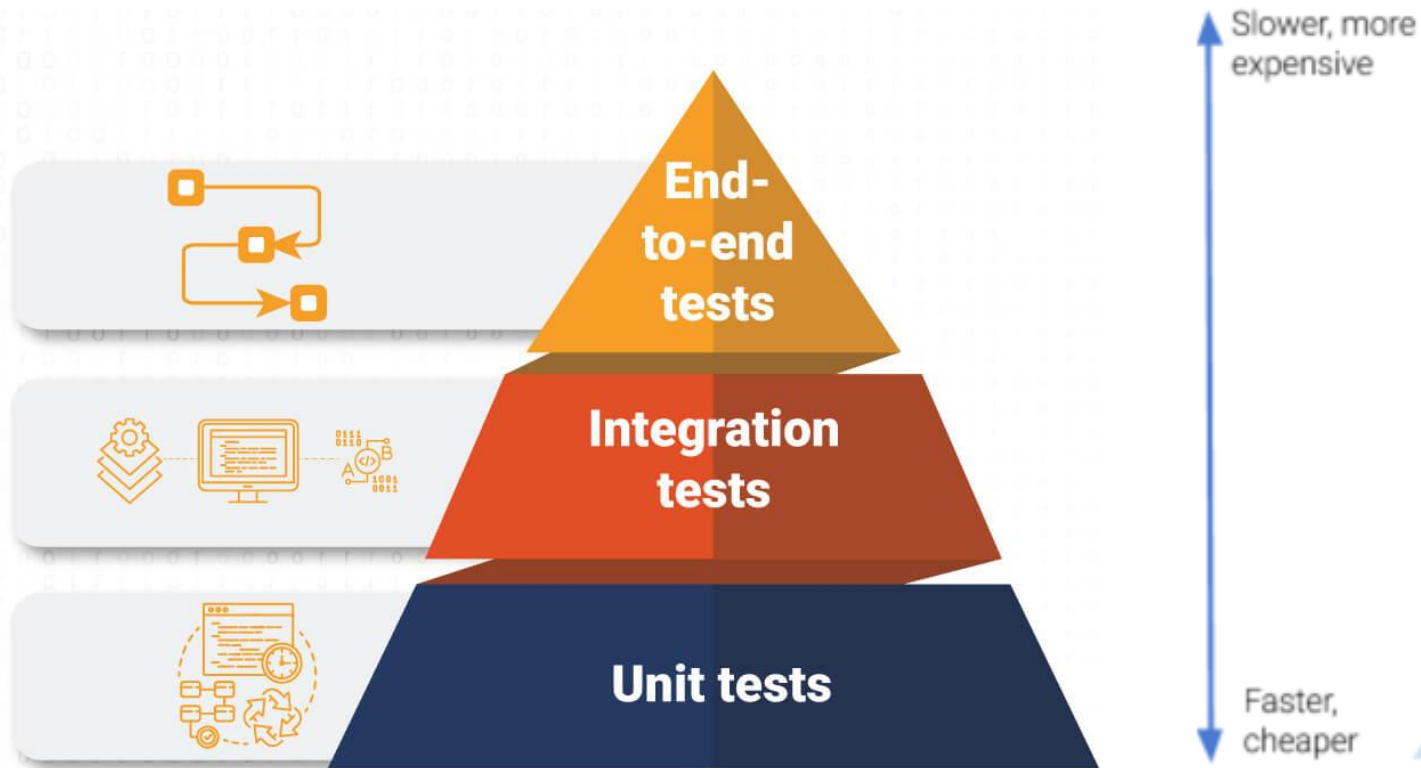


The Practical Test Pyramid

<https://martinfowler.com/articles/practical-test-pyramid.html>

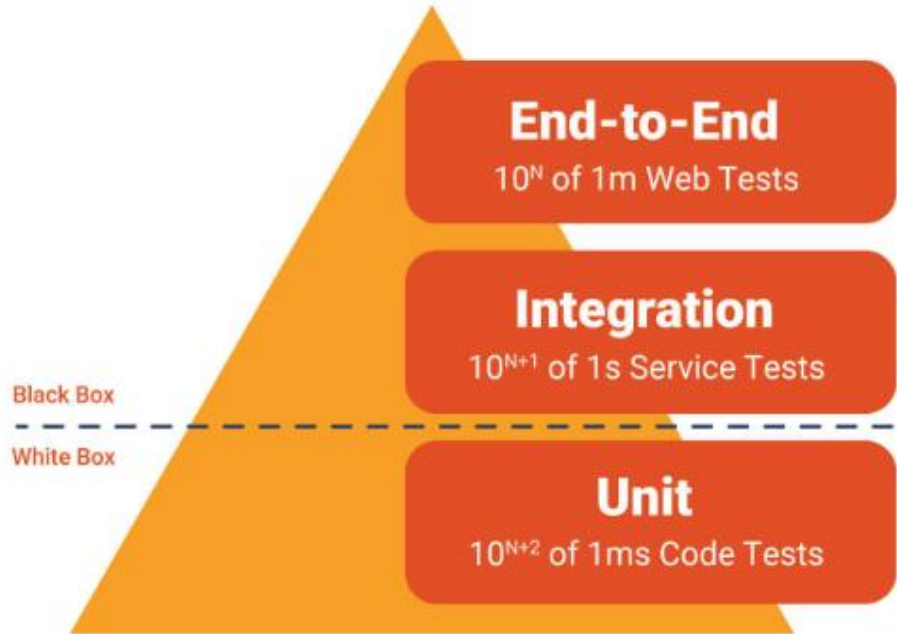


The Practical Test Pyramid



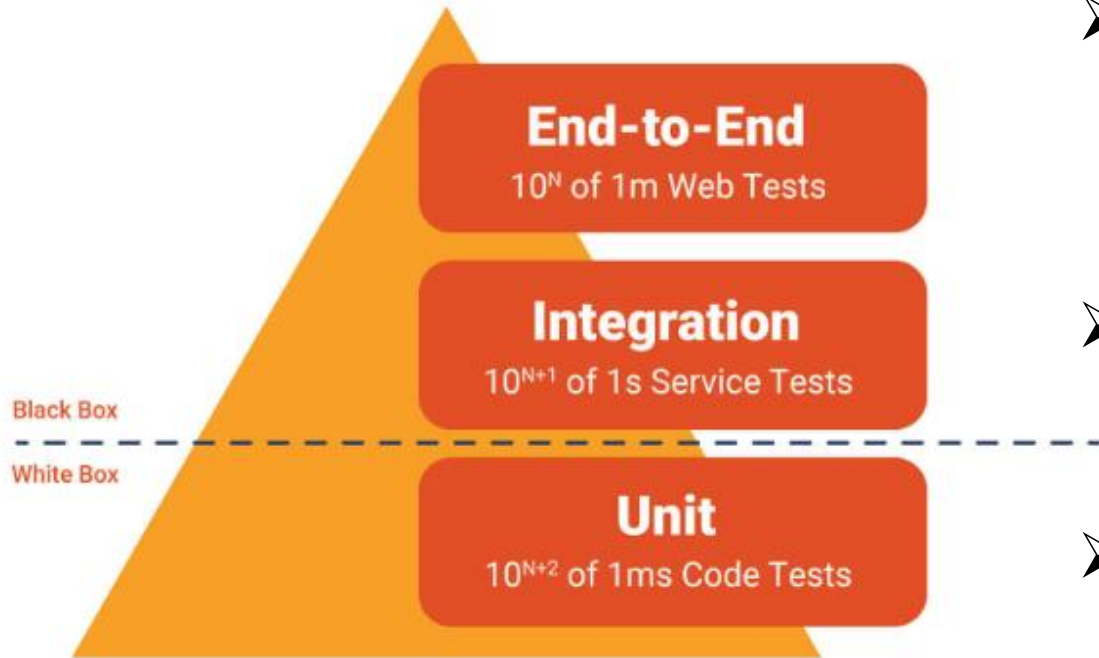
- It's essential point is that we should have **many more low-level Unit Tests** than **high level E2E (System Testing) testing** running through a GUI.

The Practical Test Pyramid



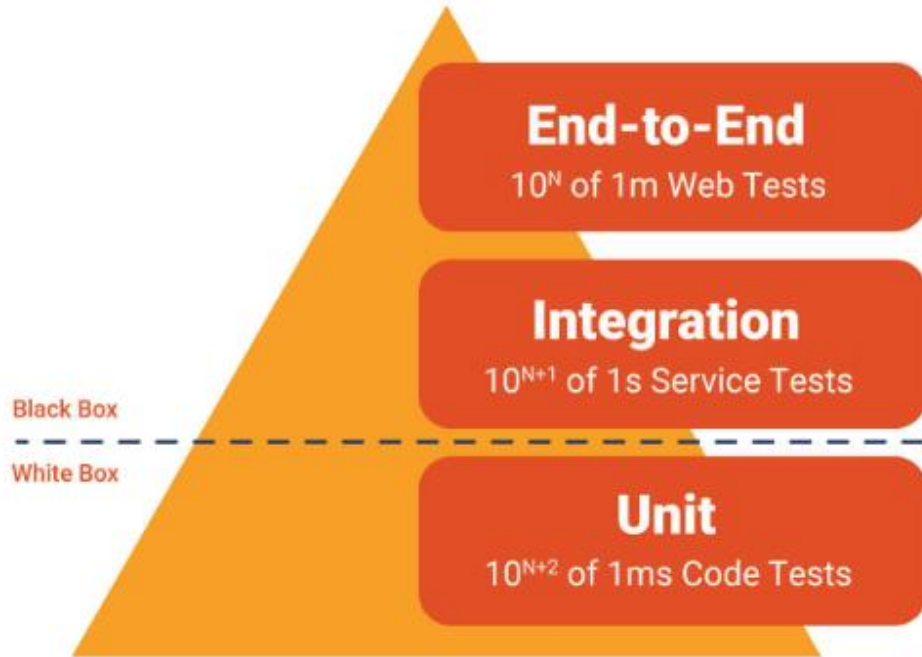
- **White Box testing** categories are mainly applied in Unit Testing. On the other hand, **Black Box testing** categories are mainly applied in Integration test and End-to-End tests.
- **Unit Test** :Unit Tests focus on the smallest unit of code, like functions or classes.
- They are short and don't have any external dependencies. If they have an external dependency, you use *mocks* instead.
- If a unit test fails, finding the issue is typically a simple process. They also have a reduced testing scope which makes them simple to write, fast to run, and easy to maintain.

The Practical Test Pyramid



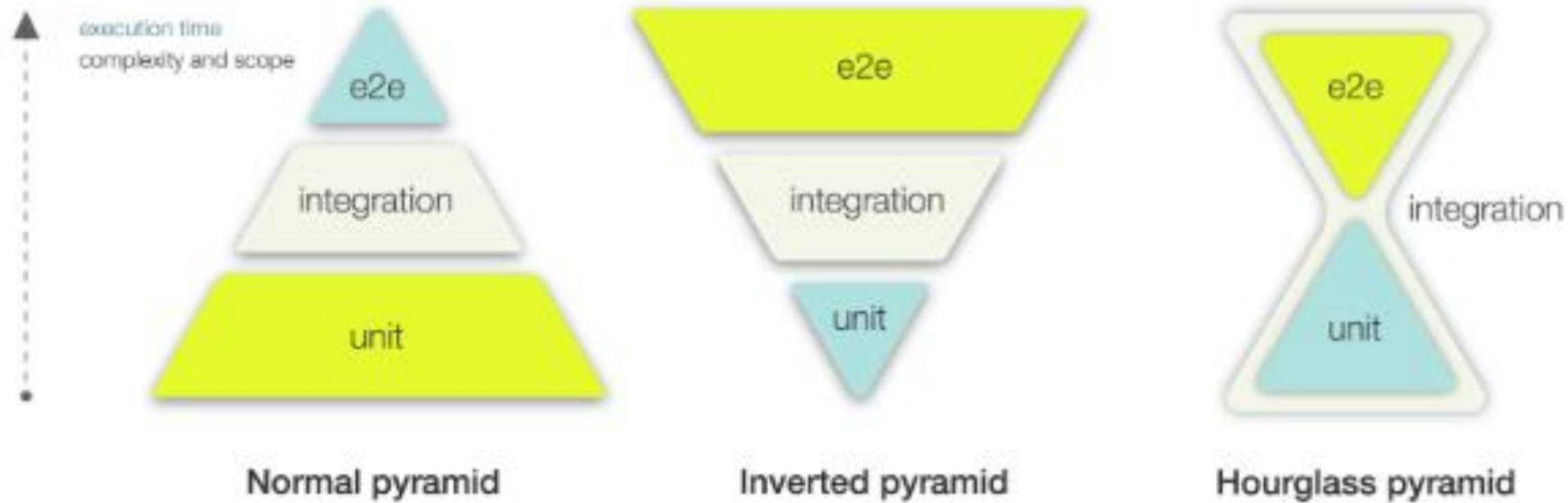
- **Integration Tests:** this focus on the interaction between two distinct entities. They are typically slower to run because more things need to be set up.
- If integration tests fail, finding the issue is a bit more challenging because the failure range is bigger.
- They are also harder to write and maintain, mostly because they need more advanced mocking and increased testing scope.

The Practical Test Pyramid



- **End-to-End (E2E) Tests:** E2E tests focus on flows, from the simplest up to the most complex. They can be viewed as a multi-step integration test.
- These tests are the slowest to run because they involve building, deploying, firing up a browser, and performing actions around the application.
- If E2E tests fail, finding the issue is often difficult because now the failure range is expanded to the entire application. Basically, along the path, anything could have broken.
- They are by far the hardest type of tests to write and maintain (from the three types considered here) because of the huge test scope and because they involve the entire application.

The Practical Test Pyramid



- A few times the pyramid will look like an inverted one, where most of the tests are E2E.
- Or it will look like an hourglass, where there are a lot of unit tests and E2E tests, but not many integration tests.

The Practical Test Pyramid



- We should follow the normal pyramid
- From the bottom to the top, each layer of testing represents **a decrease in speed** and **an increase in scope, complexity and maintenance**.
- That's why, other kinds of approaches have some problems. E2E testing is slow and costly. Compare to Unit testing , System tests(E2E tests) cover a large area of the code and require setup steps counter to unit testing which requires minimal setup focuses on a single method. Thus, E2E takes longer time to run

The Practical Test Pyramid



- Pyramid argues is **to find errors as early as possible** , that **we should do much more automated testing through unit tests** than we should through traditional GUI based testing [E2E/System test] .
- We would never want to catch something up at (System Testing) level that we could've caught down at (Unit Testing) level , and we would never want to have something bubble up to top level that we could've, for instance, caught with an integration test .
- Google suggests a 70/20/10 split: 70% unit tests, 20% integration tests, and 10% E2E tests. The exact mix will be different for each team, but in general, it should retain that pyramid shape .